

Predicting Stock Prices

with Artificial Neural Networks

Machine Learning Practice With Python

Siksha 'O' Anushandhan, ITER

Center for Data Science

Table of Contents

- 1 Demystifying Neural Networks
- 2 Implementation Frameworks

- 3 Preventing Overfitting
- 4 Stock Price Prediction Application



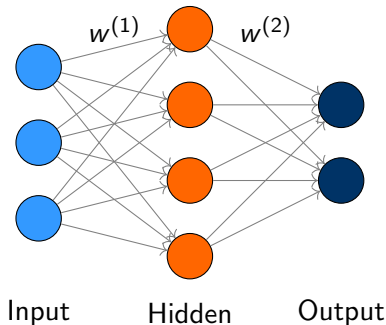
What is an Artificial Neural Network?

Definition

An **Artificial Neural Network (ANN)** is a machine learning algorithm inspired by biological neural networks, part of the broader field of Artificial Intelligence (AI).

Deep Learning

Neural networks with **two or more hidden layers** constitute **Deep Learning (DL)**



Neural Network Architecture

Three Main Components:

1 Input Layer

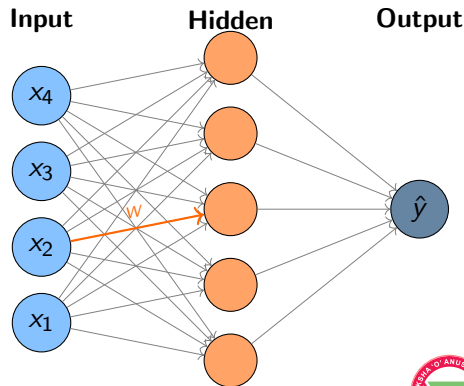
- Represents predictive features (x)
- No computation, just data entry

2 Hidden Layer(s)

- Extract latent information
- Multiple layers = Deep Learning
- Connected by **weights** (W)

3 Output Layer

- Binary: 1 node
- Multiclass: n nodes
- Regression: 1 node (prediction)



Activation Functions: Adding Non-Linearity

Why Activation Functions?



Activation Functions: Adding Non-Linearity

Why Activation Functions?

Without non-linear activation functions, stacked layers would behave as a single-layer perceptron (just linear operations).

Sigmoid (Logistic)

Range: $(0, 1)$

Use: Output layer (binary classification)

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Tanh

Range: $(-1, 1)$

Use: Hidden layers

Advantage: Steeper gradients than sigmoid

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

ReLU

Range: $(0, +\infty)$

Use: Default for hidden layers

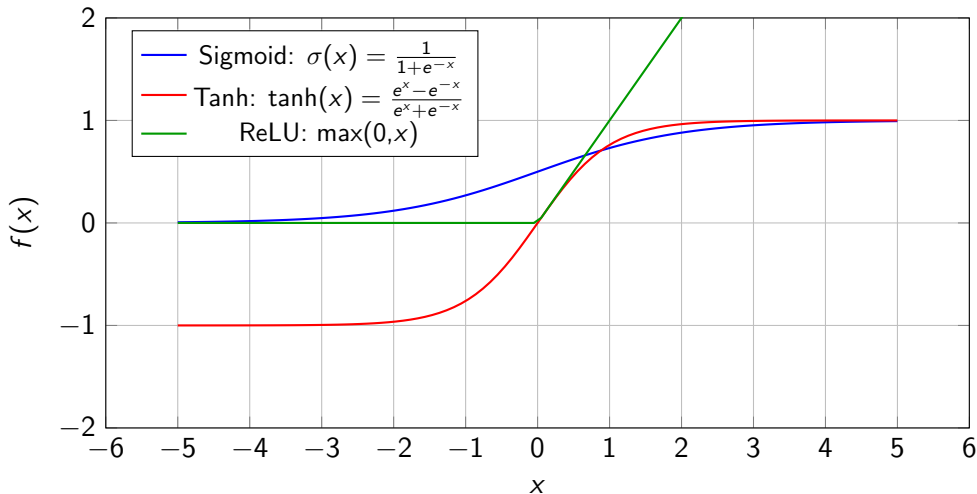
Benefits: Sparsity, mitigates vanishing gradient

$$\text{ReLU}(x) = \max(0, x)$$



Activation Functions Comparison

Sigmoid, Tanh, and ReLU Activation Functions



Training: Backpropagation Algorithm

Goal: Find optimal weights (W) that minimize the cost function (loss)

Process:

1 Forward Pass

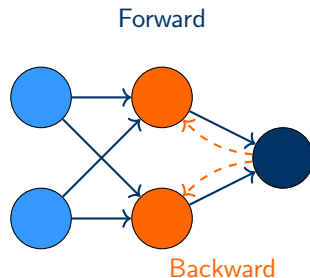
- Compute outputs for all layers
- Calculate prediction

2 Backward Pass

- Compute loss/error
- Propagate error backward
- Calculate gradients layer-by-layer

3 Update Weights

- Use gradient descent
- Adjust weights: $W \leftarrow W - \eta \nabla W$



Method: Gradient Descent

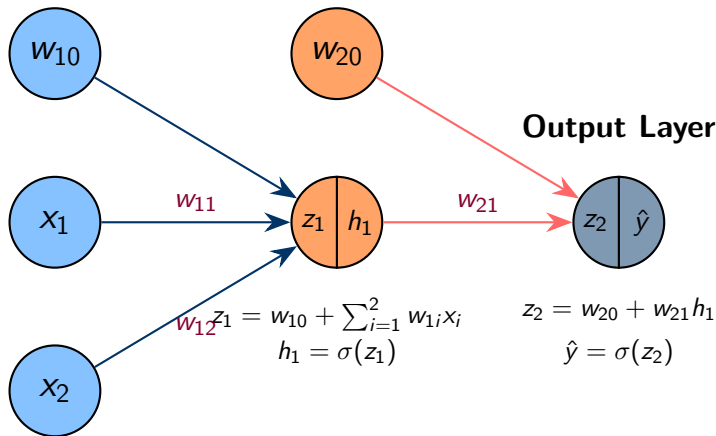


Network Architecture

Input Layer

Hidden Layer

Output Layer



Mean Square Error

$$L = \frac{1}{2}(y - \hat{y})^2$$

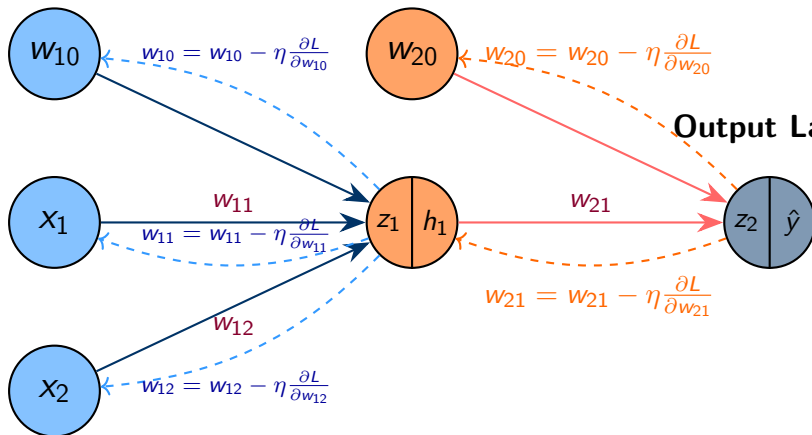


Backpropagation

Input Layer

Hidden Layer

Output Layer



Backpropagation

Output layer

Gradient for Output Weights

$$\frac{\partial L}{\partial w_{20}} = \frac{\partial L}{\partial z_2} \frac{\partial z_2}{\partial w_{20}} = \delta_2$$

$$\frac{\partial L}{\partial w_{21}} = \frac{\partial L}{\partial z_2} \frac{\partial z_2}{\partial w_{21}} = \delta_2 \cdot h_1$$

$$\begin{aligned}\delta_2 &= \frac{\partial L}{\partial z_2} \\ &= (\hat{y} - y) \cdot \sigma'(z_2) \\ &= (\hat{y} - y) \sigma(z_2) (1 - \sigma(z_2)) \\ &= (\hat{y} - y) \hat{y} (1 - \hat{y})\end{aligned}$$

Hidden layer

Gradient for hidden layer

$$\begin{aligned}\frac{\partial L}{\partial w_{1i}} &= \frac{\partial L}{\partial z_2} \frac{\partial z_2}{\partial h_1} \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial w_{1i}} \\ &= \delta_2 \cdot w_{21} \cdot \sigma'(z_1) \cdot x_i \\ &= \delta_1 \cdot x_i \quad \text{for } i = 1, 2\end{aligned}$$

$$\frac{\partial L}{\partial w_{10}} = \delta_1$$

$$\begin{aligned}\delta_1 &= \frac{\partial L}{\partial z_2} \frac{\partial z_2}{\partial h_1} \frac{\partial h_1}{\partial z_1} \\ &= \delta_2 \cdot w_{21} \cdot \sigma'(z_1) \\ &= \delta_2 \cdot w_{21} \cdot \sigma(z_1) \cdot (1 - \sigma(z_1)) \\ &= \delta_2 \cdot w_{21} \cdot h_1 \cdot (1 - h_1)\end{aligned}$$

Weight Update: Gradient Descent

Update Rule

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{\partial L}{\partial w}$$

All Weight Updates

$$w_{2i} \leftarrow w_{2i} - \eta \cdot \frac{\partial L}{\partial w_{2i}}$$

$$w_{1i} \leftarrow w_{1i} - \eta \cdot \frac{\partial L}{\partial w_{1i}}$$

where η is the **learning rate** (controls step size)

Exercise:

Draw the neural network architecture with one hidden layer for three inputs, two hidden neurons, and two output neurons. Use the ReLU activation function in the hidden layer and the sigmoid activation function in the output layer. Explain each step of the forward pass and backpropagation mechanism, detailing how the weights and biases will be updated.

Numerical Example: Setup

Given Parameters

- **Inputs:** $x_1 = 0.5$, $x_2 = 0.8$
- **Weights:** $w_{11} = 0.4$, $w_{12} = 0.6$, $w_{21} = 0.9$
- **Biases:** $w_{10} = 0.1$, $w_{20} = 0.2$
- **True output:** $y = 0.9$
- **Learning rate:** $\eta = 0.5$

Let's compute the forward pass and backpropagation!



Numerical Example: Forward Pass

Hidden Layer Computation

$$\begin{aligned} z_1 &= w_{10} + w_{11}x_1 + w_{12}x_2 \\ &= 0.1 + (0.4 \times 0.5) + (0.6 \times 0.8) \\ &= \boxed{0.78} \end{aligned}$$

$$h_1 = \sigma(0.78) = \frac{1}{1 + e^{-0.78}} \approx \boxed{0.686}$$

Output Layer Computation

$$\begin{aligned} z_2 &= w_{20} + w_{21}h_1 \\ &= 0.2 + (0.9 \times 0.686) \\ &= \boxed{0.817} \end{aligned}$$

$$\hat{y} = \sigma(0.817) = \frac{1}{1 + e^{-0.817}} \approx \boxed{0.694}$$

Loss



Numerical Example: Forward Pass

Hidden Layer Computation

$$\begin{aligned} z_1 &= w_{10} + w_{11}x_1 + w_{12}x_2 \\ &= 0.1 + (0.4 \times 0.5) + (0.6 \times 0.8) \\ &= \boxed{0.78} \end{aligned}$$

$$h_1 = \sigma(0.78) = \frac{1}{1 + e^{-0.78}} \approx \boxed{0.686}$$

Output Layer Computation

$$\begin{aligned} z_2 &= w_{20} + w_{21}h_1 \\ &= 0.2 + (0.9 \times 0.686) \\ &= \boxed{0.817} \end{aligned}$$

$$\hat{y} = \sigma(0.817) = \frac{1}{1 + e^{-0.817}} \approx \boxed{0.694}$$

Loss

$$L = \frac{1}{2}(y - \hat{y})^2 = \frac{1}{2}(0.9 - 0.694)^2 \approx \boxed{0.0212}$$



Numerical Example: Backpropagation

Output Layer

$$\begin{aligned}\delta_2 &= (\hat{y} - y) \cdot \sigma'(z_2) \\ &= (0.694 - 0.9)(0.212) \\ &= (-0.206)(0.212) \approx \boxed{-0.0437}\end{aligned}$$

Output Layer Gradient

$$\begin{aligned}\frac{\partial L}{\partial w_{20}} &= \delta_2 \approx \boxed{-0.0437} \\ \frac{\partial L}{\partial w_{21}} &= \delta_2 \cdot h_1 = (-0.0437)(0.686) \\ &\approx \boxed{-0.0300}\end{aligned}$$

Hidden Layer

$$\begin{aligned}\delta_1 &= \delta_2 \cdot w_{21} \cdot \sigma'(z_1) \\ &= (-0.0437)(0.9)(0.215) \approx \boxed{-0.00845}\end{aligned}$$

Hidden Layer Gradient

$$\begin{aligned}\frac{\partial L}{\partial w_{10}} &= \delta_1 \approx \boxed{-0.00845} \\ \frac{\partial L}{\partial w_{11}} &= \delta_1 \cdot x_1 = (-0.00845)(0.5) \\ &\approx \boxed{-0.00423} \\ \frac{\partial L}{\partial w_{12}} &= \delta_1 \cdot x_2 = (-0.00845)(0.8) \\ &\approx \boxed{-0.00676}\end{aligned}$$

Numerical Example: Weight Updates

Updated Weights

Output Layer

$$\begin{aligned}w_{20} &= w_{20} - \eta \cdot \frac{\partial L}{\partial w_{20}} \\&= 0.2 - 0.5(-0.0437) = \boxed{0.22185}\end{aligned}$$

$$\begin{aligned}w_{21} &= w_{21} - \eta \cdot \frac{\partial L}{\partial w_{21}} \\&= 0.9 - 0.5(-0.03) = \boxed{0.915}\end{aligned}$$

Hidden Layer

$$\begin{aligned}w_{10} &= w_{10} - \eta \frac{\partial L}{\partial w_{10}} \\&= 0.1 - 0.5(-0.00845) \approx \boxed{0.104225}\end{aligned}$$

$$\begin{aligned}w_{11} &= w_{11} - \eta \frac{\partial L}{\partial w_{11}} \\&= 0.4 - 0.5(-0.00423) \approx \boxed{0.402}\end{aligned}$$

$$\begin{aligned}w_{12} &= w_{12} - \eta \frac{\partial L}{\partial w_{12}} \\&= 0.6 - 0.5(-0.00676) \approx \boxed{0.603}\end{aligned}$$

The weights have been updated to reduce the loss! Repeat this process for many iterations to train the network.

Deep Learning: Stacking More Layers

Definition

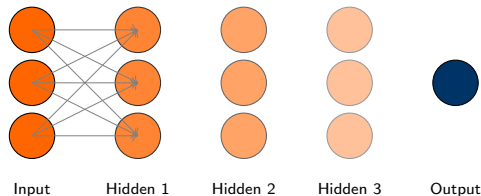
Deep Neural Network:
Contains 2+ hidden layers

Advantages:

- **Hierarchical feature extraction**
- Better learning of complex non-linear relationships
- Superior to shallow networks for complex tasks

Major Breakthroughs:

- **Computer Vision:** Face recognition, image generation
- **NLP:** Machine translation, text generation



**Features extracted hierarchically
through stacked layers**



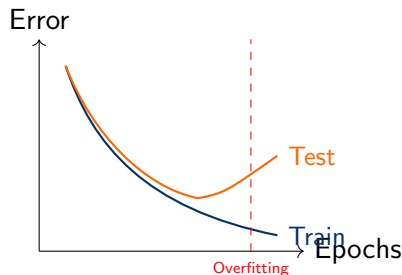
Overfitting: The Challenge

Problem

Neural networks can fit training data **too strictly**, memorizing patterns rather than learning generalizable features.

Result:

- Excellent training performance
- Poor generalization
- High test error



Solution 1: Dropout

Mechanism

Randomly **deactivate** a fraction of hidden nodes during each training iteration

Benefits:

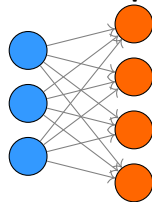
- Prevents memorization of co-dependencies
- Forces network to learn robust features
- Acts as ensemble learning
- Reduces overfitting effectively

Important

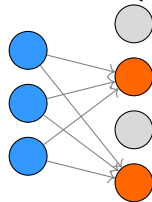
Dropout rate must be **carefully fine-tuned**

Typical values: 0.2 - 0.5

Without Dropout:



With Dropout (50%):



Solution 2: Early Stopping

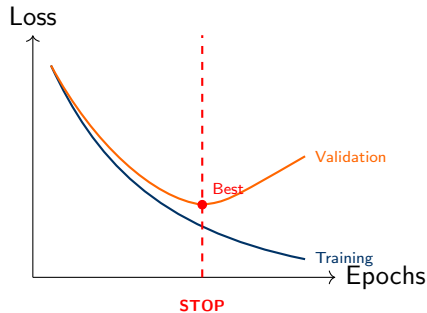
Mechanism

Monitor validation performance during training and **stop** when performance degrades

Process:

- 1 Split data: Train / Validation / Test
- 2 Train model while monitoring validation loss
- 3 Track best validation performance
- 4 Stop when validation loss increases
- 5 Restore best model weights

Benefit: Controls model complexity automatically



Stop training when validation loss starts increasing



Applying Neural Networks to Stock Prediction

Load 'SBI' stock data by using `yfinance`, and do all necessary feature engineering, removing NaN values, and split into train and test data. (**As like Chapter 5**)



Applying Neural Networks to Stock Prediction

Load 'SBI' stock data by using `yfinance`, and do all necessary feature engineering, removing NaN values, and split into train and test data. (**As like Chapter 5**)

Build neural network using `scikit-learn`

```
from sklearn.neural_network import MLPRegressor

MLPRegressor(hidden_layer_sizes=(100,), activation='relu',
*, solver='adam', alpha=0.0001, batch_size='auto',
learning_rate='constant', learning_rate_init=0.001,
power_t=0.5, max_iter=200, shuffle=True, random_state=None,
tol=0.0001, verbose=False, warm_start=False, momentum=0.9,
nesterovs_momentum=True, early_stopping=False,
validation_fraction=0.1, beta_1=0.9, beta_2=0.999, epsilon=1e-08,
n_iter_no_change=10, max_fun=15000)
```

Applying Neural Networks to Stock Prediction

<code>hidden_layer_sizes</code>	The i-th element represents the number of neurons in the i-th hidden layer.
<code>activation</code>	'identity', 'logistic', 'tanh', 'relu'
<code>solver</code>	'lbfgs', 'sgd', 'adam'
<code>batch_size</code>	If the solver is 'lbfgs', the regressor will not use minibatch. When set to "auto", 'batch_size=min(200, n_samples)'.
<code>verbose</code>	To print progress message
<code>warm_start</code>	When set to True, reuse the solution of the previous call to fit as initialization, otherwise, just erase the previous solution.
<code>epsilon</code>	Value for numerical stability in adam. Only used when solver='adam'.
<code>max_fun</code>	Only used when solver='lbfgs'. Maximum number of function calls. The solver iterates until number of iterations reaches max_iter, or this number of function calls. Note that number of function calls will be greater than or equal to the number of iterations.



Applying Neural Networks to Stock Prediction

- Build a neural network using `MLPRegressor` with two hidden layers as 1st hidden layer containing 16 neurons and 2nd hidden layer containing 8 neurons, ReLU activation, adam optimizer, early stopping, and print progress message at each iteration of training.



Applying Neural Networks to Stock Prediction

- Build a neural network using MLPRegressor with two hidden layers as 1st hidden layer containing 16 neurons and 2nd hidden layer containing 8 neurons, ReLU activation, adam optimizer, early stopping, and print progress message at each iteration of training.

```
1 nn_scikit = MLPRegressor(hidden_layer_sizes=[16,8],  
    activation= 'relu', solver= 'adam',  
    learning_rate_init= 0.001, max_iter= 1000,  
    early_stopping= True, verbose= True, random_state=42)
```

- Train the model and print MSE



Applying Neural Networks to Stock Prediction

- Build a neural network using MLPRegressor with two hidden layers as 1st hidden layer containing 16 neurons and 2nd hidden layer containing 8 neurons, ReLU activation, adam optimizer, early stopping, and print progress message at each iteration of training.

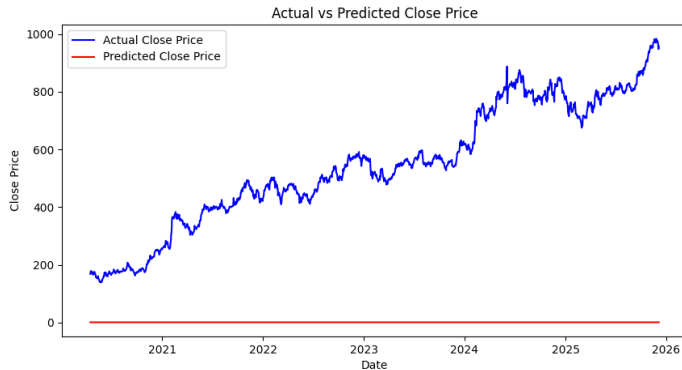
```
1 nn_scikit = MLPRegressor(hidden_layer_sizes=[16,8],  
    activation= 'relu', solver= 'adam',  
    learning_rate_init= 0.001, max_iter= 1000,  
    early_stopping= True, verbose= True, random_state=42)
```

- Train the model and print MSE

```
1 nn_scikit.fit(X_train, Y_train)  
2 Y_pred = nn_scikit.predict(X_test)  
3 print('MSE:', mean_squared_error(Y_test, Y_pred))
```



Plot actual vs prediction



PREDICTION IS
VERY BAD



Applying Neural Networks to Stock Prediction

- Standardize all feature value, and then train the model, predict, and plot actual vs prediction



Applying Neural Networks to Stock Prediction

- Standardize all feature value, and then train the model, predict, and plot actual vs prediction

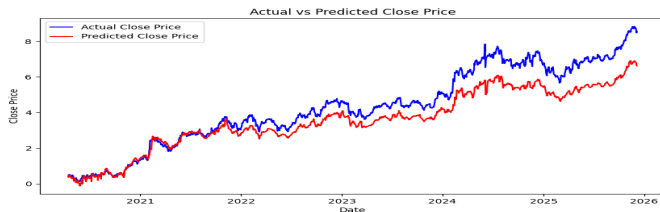
```
1 from sklearn.preprocessing import StandardScaler
2 scaler = StandardScaler()
3 X_train_scaled = scaler.fit_transform(X_train)
4 X_test_scaled = scaler.transform(X_test)
5 Y_train_scaled =
    scaler.fit_transform(Y_train.values.reshape(-1,1))
6 Y_test_scaled = scaler.transform(Y_test.values.reshape(-1,1))
```



Applying Neural Networks to Stock Prediction

- Standardize all feature value, and then train the model, predict, and plot actual vs prediction

```
1 from sklearn.preprocessing import StandardScaler
2 scaler = StandardScaler()
3 X_train_scaled = scaler.fit_transform(X_train)
4 X_test_scaled = scaler.transform(X_test)
5 Y_train_scaled =
    scaler.fit_transform(Y_train.values.reshape(-1,1))
6 Y_test_scaled = scaler.transform(Y_test.values.reshape(-1,1))
```



Prediction is not in original scale



Applying Neural Networks to Stock Prediction

- Make the prediction into original scale and plot actual vs prediction.



Applying Neural Networks to Stock Prediction

- Make the prediction into original scale and plot actual vs prediction.

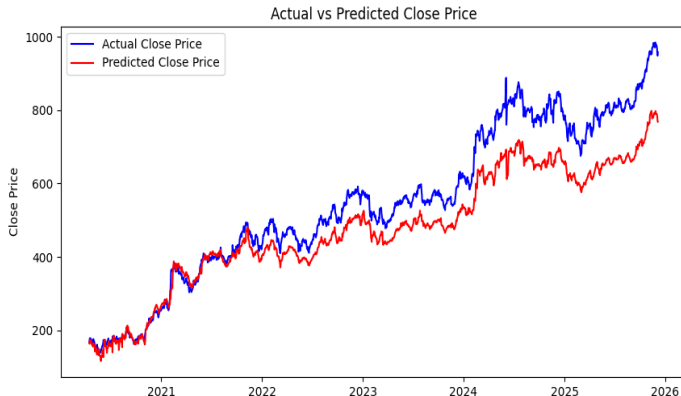
```
1 y_pred_actual =  
    scaler.inverse_transform(Y_pred_scaled.reshape(-1, 1))
```



Applying Neural Networks to Stock Prediction

- Make the prediction into original scale and plot actual vs prediction.

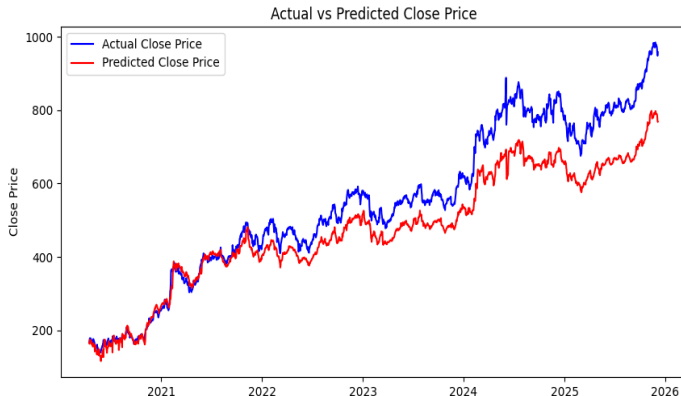
```
1 y_pred_actual =  
    scaler.inverse_transform(Y_pred_scaled.reshape(-1, 1))
```



Applying Neural Networks to Stock Prediction

- Make the prediction into original scale and plot actual vs prediction.

```
1 y_pred_actual =  
    scaler.inverse_transform(Y_pred_scaled.reshape(-1, 1))
```



Use different hidden_layer_sizes, activation, solver, and other parameters to reduce MSE



Building neural network using TensorFlow

- import the necessary modules and set a random seed, which is recommended for reproducible modeling:

```
1 import tensorflow as tf
2 from tensorflow import keras
3 tf.random.set_seed(42)
```

- Create a Keras Sequential model by passing a list of layer instances to the constructor, including two fully connected hidden layers with 16 nodes and 8 nodes, respectively. And again, ReLU activation is used:

```
1 model = keras.Sequential([
2     keras.layers.Dense(16, activation='relu',
3                         input_shape=(X_train_scaled.shape[1],)),
4     keras.layers.Dense(8, activation='relu'),
5     keras.layers.Dense(1)
6 ])
# keras.layers.Dense(1, activation='relu') # added ReLU on
# output for non-negative predictions
```

Building neural network using TensorFlow

- Compile the model by using Adam as the optimizer

```
1 model.compile(optimizer='adam', loss='mean_squared_error')
2 # model.compile(optimizer=keras.optimizers.Adam( learning_rate
    = 0.001), loss='mean_squared_error')
```



Building neural network using TensorFlow

- Compile the model by using Adam as the optimizer

```
1 model.compile(optimizer='adam', loss='mean_squared_error')
2 # model.compile(optimizer=keras.optimizers.Adam( learning_rate
    = 0.001), loss='mean_squared_error')
```

- Fit the model.

```
1 model.fit(X_train_scaled, Y_train_scaled, epochs=100,
    batch_size=32, validation_split=0.2, verbose=1)
2 # history = model.fit(X_train_scaled, Y_train_scaled,
    epochs=100, batch_size=32, validation_split=0.2,
    verbose=1)
```



Building neural network using TensorFlow

- Compile the model by using Adam as the optimizer

```
1 model.compile(optimizer='adam', loss='mean_squared_error')
2 # model.compile(optimizer=keras.optimizers.Adam( learning_rate
    = 0.001), loss='mean_squared_error')
```

- Fit the model.

```
1 model.fit(X_train_scaled, Y_train_scaled, epochs=100,
    batch_size=32, validation_split=0.2, verbose=1)
2 # history = model.fit(X_train_scaled, Y_train_scaled,
    epochs=100, batch_size=32, validation_split=0.2,
    verbose=1)
```

- Predict for test data

```
1 Y_pred_tf_scaled = model.predict(X_test_scaled)
```



Building neural network using TensorFlow

- Compile the model by using Adam as the optimizer

```
1 model.compile(optimizer='adam', loss='mean_squared_error')
2 # model.compile(optimizer=keras.optimizers.Adam( learning_rate
    = 0.001), loss='mean_squared_error')
```

- Fit the model.

```
1 model.fit(X_train_scaled, Y_train_scaled, epochs=100,
    batch_size=32, validation_split=0.2, verbose=1)
2 # history = model.fit(X_train_scaled, Y_train_scaled,
    epochs=100, batch_size=32, validation_split=0.2,
    verbose=1)
```

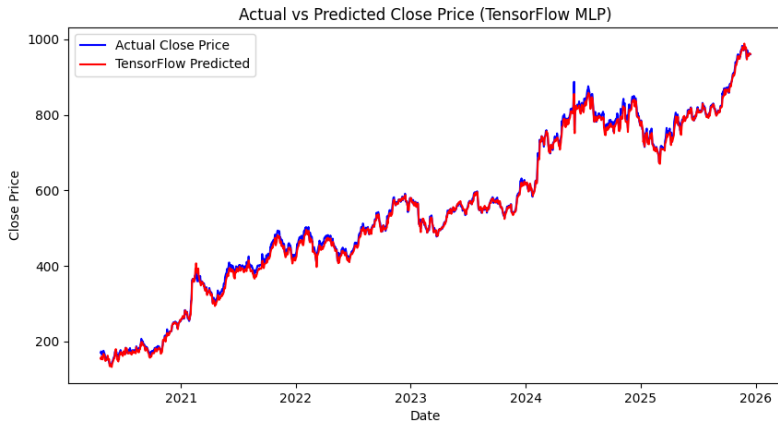
- Predict for test data

```
1 Y_pred_tf_scaled = model.predict(X_test_scaled)
```



Building neural network using TensorFlow

- Plot actual vs predicted in original scale.



Training and validation loss

- Plot training and validation loss for each epoch.

```
1 loss = history.history.get('loss')
2 val_loss = history.history.get('val_loss')
3 plt.figure(figsize=(10, 5))
4 epochs = range(1, len(loss) + 1)
5 plt.plot(epochs, loss, label='Training Loss', color='C0')
6 if val_loss is not None:
7     plt.plot(epochs, val_loss, label='Validation Loss',
              color='C1')
```



Neural Network with dropout

- Neural network with dropout

```
1 model = keras.Sequential([
2     keras.layers.Dense(16, activation='relu',
3         input_shape=(X_train_scaled.shape[1],)),
4     keras.layers.Dropout(0.2),
5     keras.layers.Dense(8, activation='relu'),
6     keras.layers.Dropout(0.2),
7     keras.layers.Dense(1)
8 ])
```

Training and predicting are same as previous



Neural Network with early stopping

- Neural network with early stopping

```
1 model = keras.Sequential([
2     keras.layers.Dense(16, activation='relu',
3         input_shape=(X_train_scaled.shape[1],)),
4     keras.layers.Dense(8, activation='relu'),
5     keras.layers.Dropout(0.2),
6     keras.layers.Dense(1)
7 ])
8 early_stop = keras.callbacks.EarlyStopping(monitor='val_loss',
9     patience=10, restore_best_weights=True, verbose=1)
10 model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001),
11     loss='mean_squared_error')
12 history = model.fit(X_train_scaled, Y_train_scaled,
13     epochs=200, batch_size=32, validation_split=0.2,
14     callbacks=[early_stop], verbose=1)
```